

CSE 141L Milestone 3

Daniel Namkung

May 2026

Academic Integrity

Your work will not be graded unless the signatures of all members of the group are present beneath the honor code.

To uphold academic integrity, students shall:

- Complete and submit academic work that is their own and that is an honest and fair representation of their knowledge and abilities at the time of submission.
- Know and follow the standards of CSE 141L and UCSD.

Please sign (type) your name(s) below the following statement:

I pledge to be fair to my classmates and instructors by completing all of my academic work with integrity. This means that I will respect the standards set by the instructor and institution, be responsible for the consequences of my choices, honestly represent my knowledge and abilities, and be a community member that other can trust to do the right thing even when no one is watching. I will always put learning before grades, and integrity before performance. I pledge to excel with integrity.

Daniel Namkung

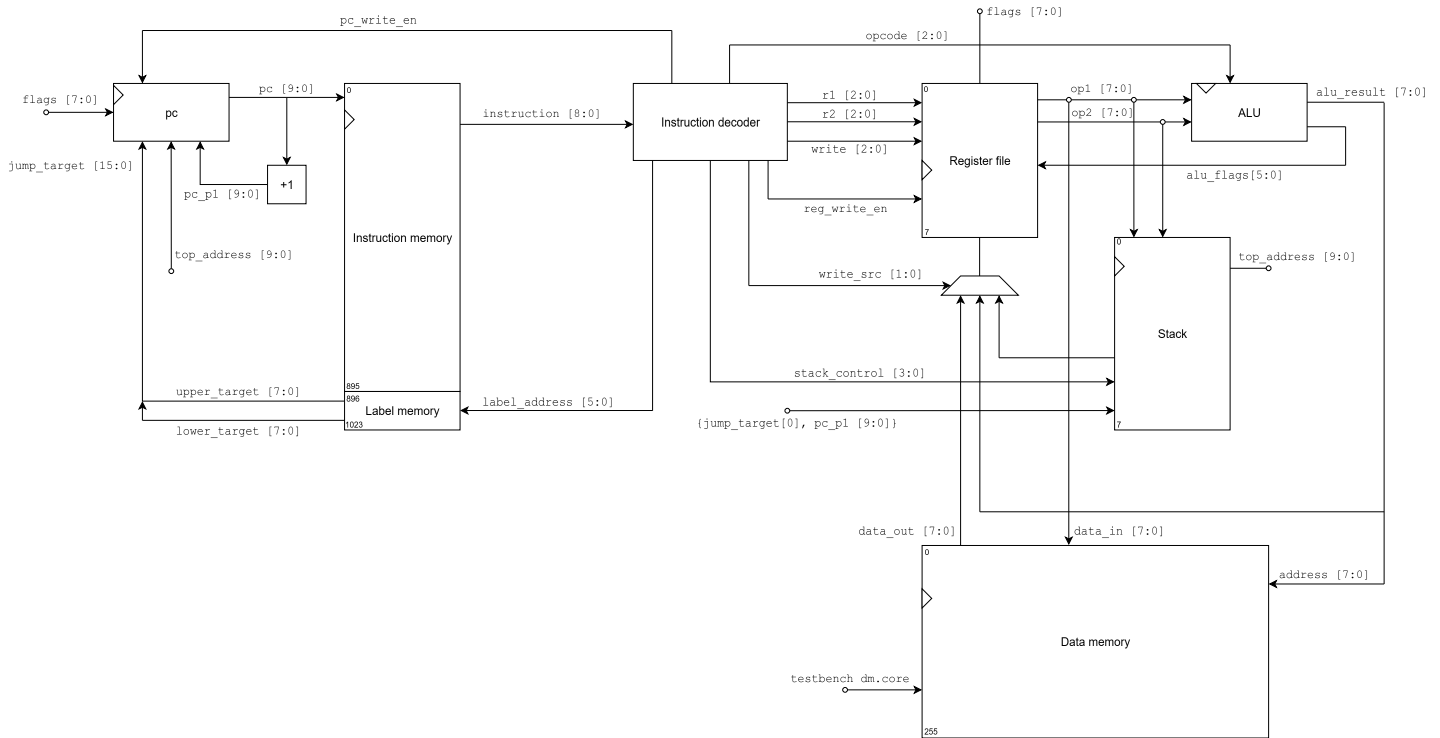
0 Team

- Daniel Namkung

1 Introduction

- i. The name of this architecture is 'Sisyphus.' Its name reflects the general approach to the architecture's operation and also how I felt while writing the assembly, assembler, and simulator every time that a change was made to the ISA. When designing, the primary goal was supporting function calls with a stack using just the 9-bit instruction width limit, which also helps with minimizing the overall memory needed to run the programs.
- ii. Data needs to be loaded into a register before an operation can be performed on it, so it falls into the load-store category. The zero register also serves as an accumulator and most values resulting from a calculation are stored there. Many instructions also use the zero register as a source for one of their operands.

2 Architectural Overview



3 Machine Specification

i. Instruction formats:

Type	Instructions	Description
J	jump, jumplt, jumpgt, jumplt, jumpgt, jumpz, jumpnz, jumppos, call, return	Modifies the program counter
R	add, imm, copy, lshift, rshift	Performs basic register operations
Z	popcnt, not, xor, and, addc, sub, clshift	Performs register operations using r0 as one of the operands
M	load, store	Interacts with data memory
S	push, pop	Interacts with the stack
C	cmp	Performs a comparison
H	halt	Stops the current program

ii. Operations:

jump <target>

Unconditionally jumps to the specified target in the program

```
#start
    jump #new-position
    ...

    #new-position //jumps here
```

Format | Opcode 001 | Address (6 bits) |

- Address specifies one of 63 program addresses stored in memory

jumplt <target>

Jumps to the specified target in the program if the `less_than_unsigned` flag was set to true by an unsigned comparison operation

```

#start
    imm r1 4
    imm r0 3
    cmp r0 r1 //less_than becomes TRUE since r0 < r1
    jumplt #new-position
    ...

    #new-position //jumps here

```

Format | Opcode 001 | Address (6 bits) |

- Address specifies one of 63 program addresses stored in memory
-

jumpgt <target>

Jumps to the specified target in the program if the **greater_than_unsigned** flag was set to true by an unsigned comparison operation

```

#start
    imm r1 3
    imm r0 4
    cmp r0 r1 //greater_than becomes TRUE since r0 > r1
    jumplt #new-position
    ...

    #new-position //jumps here

```

Format | Opcode 001 | Address (6 bits) |

- Address specifies one of 63 program addresses stored in memory
-

jumplt <target>

Jumps to the specified target in the program if the **less_than_signed** flag was set to true by a signed comparison operation

```

#start
    imm r1 1
    imm r0 -1
    cmp r0 r1 //less_than becomes TRUE since r0 < r1
    jumplt #new-position
    ...

    #new-position //jumps here

```

Format | Opcode 001 | Address (6 bits) |

- Address specifies one of 63 program addresses stored in memory
-

jumpgt <target>

Jumps to the specified target in the program if the **greater_than_signed** flag was set to true by a signed comparison operation

```

#start
    imm r1 -1
    imm r0 1
    cmp r0 r1 //greater_than becomes TRUE since r0 > r1
    jumplt #new-position
    ...

    #new-position //jumps here

```

Format | Opcode 001 | Address (6 bits) |

- Address specifies one of 63 program addresses stored in memory
-

jumpz <target>

Jumps to the specified target in the program if the current value in **r0** is zero.

```

#start
    imm r1 -1
    imm r2 1
    add r1 r2
    jumpz #new-position
    ...

    #new-position //jumps here

```

Format | Opcode 001 | Address (6 bits) |

- Address specifies one of 63 program addresses stored in memory
-

jumpnz <target>

Jumps to the specified target in the program if the current value in **r0** is nonzero.

```

#start
    imm r1 3
    imm r2 1
    add r1 r2
    jumpnz #new-position
    ...

    #new-position //jumps here

```

Format | Opcode 001 | Address (6 bits) |

- Address specifies one of 63 program addresses stored in memory
-

jumpnz <target>

Jumps to the specified target in the program if the current value in **r0** is positive.

```

#start
    imm r1 3
    imm r2 -1
    add r1 r2
    jumpnpos #new-position
    ...

    #new-position //jumps here

```

Format | Opcode 001 | Address (6 bits) |

- Address specifies one of 63 program addresses stored in memory
-

call <function>

Calls the specified function by jumping to the first instruction in the function. Only the jump is handled by the instruction, and stack operations should be handled separately as needed.

```
#start
...
    stack push r4+
    call #my-function
    stack pop
...

#my-function //jumps here
...
    return
```

Format | Opcode 001 | Address (6 bits) |

- Address specifies one of 63 program addresses stored in memory
-

return

Reads the stored program address from the top of the stack and jumps there. This instruction does not modify the stack.

```
#start
...
    push                //address = 10, stores address = 12 to the stack.
    call #my-function   //address = 11
    pop                 //address = 12

#my-function
...
    return              //jumps to pop
```

Format | Opcode 001 | 000000 |

- This uses the reserved address 000000 which indicates that the new program counter should be read from the stack instead of instruction memory.
-

imm <destination> <immediate>

Loads an immediate value into the destination register. The immediate value can be in the range 1 - 6 (inclusive) or -1. Use **xor** and **copy** to load zeroes.

```
#start
    imm r1 1
    imm r2 2
    imm r3 3
    imm r4 4
    imm r5 5
    imm r6 6
    imm r7 -1
```

Format | Opcode 000 | Destination (3 bits) | Immediate (3 bits) |

- Destination specifies one of eight registers (r0 - r7)
 - Immediate specifies the immediate value being loaded (0b001 - 0b110) or 0b111 for -1.
-

copy <destination> [source]

Copies the contents of the source register into the destination register

```
#start
    imm r5 3
    copy r4 r5    //both r4 and r5 now contain 3
    lshift r4 1   //store 6 in r0
    copy r7       //copy 6 from r0 into r7
```

Format | Opcode 010 | Destination (3 bits) | Source (3 bits) |

- Destination specifies one of eight registers (r0 - r7)
 - Source specifies one of eight registers (r0 - r7) and defaults to r0 if not specified
-

popcnt <operand 1>

Calculates the population count of the bit sequence in the specified register and stores the result in r0.

```
#start
    imm r3 3    //011
    imm r4 -1   //100
    popcnt r3
    copy r5      //r5 now contains 3

    popcnt r4
    copy r6      //r6 now contains 8
```

Format | Opcode 011 | Operand 1 (3 bits) | 000 |

- Operand 1 specifies one of eight registers (r0 - r7)
-

not <operand 1>

Performs a bitwise NOT on the operands stored in the specified register and stores the result in r0

```
#start
    imm r4 3
    not r4    //r0 contains 0xFC
    imm r4 5
    lshift r4 4
    add r0 r4 //r0 contains 0x55
    not r0    //r0 contains 0xAA
```

Format | Opcode 011 | Operand 1 (3 bits) | 001 |

- Operand 1 specifies one of eight registers (r0 - r7)
-

and <operand 1>

Performs a bitwise AND on the operands stored in the specified register and stores the result in r0

```
#start
    imm r1 3
    imm r0 2
    and r1 //r0 holds 0x02
    imm r1 -1
    imm r0 4
    lshift r0 2
    add r0 r1 //r0 holds 0x0F
    and r1 //r0 holds 0x0F
```

Format | Opcode 011 | Operand 1 (3 bits) | 010 |
- Operand 1 specifies one of eight registers (r0 - r7)

xor <operand 1>

Performs a bitwise XOR on the operands stored in the specified register and stores the result in r0

```
#start
    imm r1 5
    imm r2 3

    //swap r1 and r2 with the xor trick™
    copy r0 r1
    xor r2
    copy r1 //r1 = r1 ^ r2
    copy r0 r2
    xor r1
    copy r2 //r2 = r1 ^ r2 = (r1 ^ r2) ^ r2 = r1
    copy r0 r1
    xor r2
    copy r1 //r1 = r1 ^ r2 = (r1 ^ r2) ^ r1 = r2
```

Format | Opcode 011 | Operand 1 (3 bits) | 011 |
- Operand 1 specifies one of eight registers (r0 - r7)

addc <operand 1>

Adds the operand stored in the specified register to the value stored in r0 and the current value of the carry flag. If a carry is produced, updates the carry flag to 1.

```
#start
    imm r0 -1
    lshift r0 2 //r0 holds 252
    imm r1 6
    add r0 r1 //r0 holds 4 and sets the carry flag
    imm r2 5
    addc r2    //r0 = 4 + 5 + 1 = 10
```

Format | Opcode 011 | Operand 1 (3 bits) | 100 |
- Operand 1 specifies one of eight registers (r0 - r7)

sub <operand 1>

Subtracts the operand stored in the specified register from the value stored in r0.

```
#start
    imm r0 3
    imm r1 2
    sub r1 //r0 holds 1
    sub r1 //r0 holds -1
    sub r1 //r0 holds -3
```

Format | Opcode 011 | Operand 1 (3 bits) | 101 |
- Operand 1 specifies one of eight registers (r0 - r7)

lshiftc <operand 1>

Shifts the operand stored in the specified register left by one place, and shifts in the current value of the carry flag into the least significant bit. The leftmost bit that is shifted out becomes the new carry flag.

```
#start
    imm r0 -1
    lshift r0 2 //r0 holds 252
    imm r1 6
    add r0 r1 //r0 holds 4 and sets the carry flag
    lshiftc r0 //r0 becomes 9
```

Format | Opcode 011 | Operand 1 (3 bits) | 110 |
 - Operand 1 specifies one of eight registers (r0 - r7)

add <operand 1> <operand 2>

Adds the values stored in the two specified registers and stores the result in r0. If a carry is produced, updates the carry flag to 1.

```
#start
    imm r2 3
    imm r3 5
    add r2 r3 //stores 8 in r0
    copy r2
    add r2 r3 //stores 13 in r0
```

Format | Opcode 110 | Operand 1 (3 bits) | Operand 2 (3 bits) |
 - Operand 1 specifies one of eight registers (r0 - r7)
 - Operand 2 specifies one of eight registers (r0 - r7)

lshift <operand 1> <immediate>

Logical shifts the value stored in the specified register left by the immediate value. The immediate value can be in the range 1 - 4 (inclusive). The leftmost bit that is shifted out becomes the new carry flag.

```
#start
    imm r1 2
    lshift r1 4 //stores 32 in r0
    lshift r0 2 //stores 128 in r0
    imm r0 3
    lshift r0 2 //stores 12 in r0
    lshift r0 1 //stores 24 in r0
```

Format | Opcode 101 | Operand 1 (3 bits) | 0 | Immediate (2 bits) |
 - Operand 1 specifies one of eight registers (r0 - r7)
 - Immediate specifies the immediate value to shift by (0b01 - 0b11) or 0b00 for 4.

rshift <operand 1> <immediate>

Logical shifts the value stored in the specified register right by the immediate value. The immediate value can be in the range 1 - 4 (inclusive).

```
#start
    imm r1 6
    rshift r1 2 //stores 1 in r0
    imm r1 5
    lshift r1 4
    rshift r0 3 //stores 10 in r0
```

Format | Opcode 101 | Operand 1 (3 bits) | 1 | Immediate (2 bits) |
 - Operand 1 specifies one of eight registers (r0 - r7)

- Immediate specifies the immediate value to shift by (0b01 - 0b11) or 0b00 for 4.
-

store <address> [offset]

Stores the value in **r0** at the address in the specified register. The address can be offset with the specified immediate. The immediate value can be in the range 0 - 3 (inclusive) or 0 if no immediate is specified.

```
#start
    imm r1 1
    lshift r1 4 //store address 16 in r1
    imm r3 6
    imm r4 5
    add r3 r4    //store 11 in r0
    store r1     //write 11 to address 16 in memory
    add r1 r0    //store 27 in r0
    store r1 1   v//write 27 to address 17 in memory
```

Format | Opcode 111 | Address (3 bits) | 0 | Immediate (2 bits) |

- Address specifies one of eight registers (**r0** - **r7**)
 - Immediate specifies the immediate value to shift by (0b00 - 0b11).
-

load <address> [offset]

Loads from the address stored in the specified register into **r0**. The address can be offset with the specified immediate. The immediate value can be in the range 0 - 3 (inclusive) or 0 if no immediate is specified.

```
#start
    imm r1 1
    lshift r1 4 //store address 16 in r1
    imm r0 6
    store r1     //write 6 to address 16 in memory
    ...
    imm r1 5
    lshift r1 1
    add r0 r1
    copy r7      //store address 15 in r7
    load r7 1    //load from address 16 into r0
```

Format | Opcode 111 | Address (3 bits) | 1 | Immediate (2 bits) |

- Address specifies one of eight registers (**r0** - **r7**)
 - Immediate specifies the immediate value to shift by (0b00 - 0b11).
-

push [registers]

Pushes the values in the registers **r1** - **r7** to the stack, along with which registers will be restored in the next **pop**. The program address **pc + 2** is also pushed. This also creates the requirement that **push** executed immediately before a **call**. The register groups that can be saved are **r2+**, **r4+**, **r5+**, and **r6+**. A group **rN+** will save the registers **rN** through **r7** (inclusive).

```

#start
    imm r3 5
    imm r4 6
    imm r5 2
    push r4+ //registers r4-r7 are saved to the stack with the program counter
    call #my-function
    pop      //registers r4-r7 are restored, r3 remains at 2

#my-function
...
    imm r3 2
    imm r4 1
    return

```

Format | Opcode 000 | 0 | Registers (2 bits) | 000 |

- Registers specifies the range of registers to be restored in the next `pop` with the encoding `r6+` → 11, `r5+` → 10, `r4+` → 01, and `r2+` → 00.
-

`pop`

Restores the register values from the frame at the top of the stack, and removes the corresponding frame. The registers specified when the frame was pushed determine which registers will be restored from the values saved in the stack.

```

#start
    imm r4 6
    imm r5 2
    imm r6 3
    imm r7 4
    push r5+ //save values to stack to keep more than 8 registers
    imm r5 1
    imm r6 2
    add r5 r6
    pop      //r5-r7 are restored

```

Format | Opcode 000 | 1 | Registers (2 bits) | 000 |

- Registers specifies the range of registers to be restored in the next `pop` with the encoding `r6+` → 11, `r5+` → 10, `r4+` → 01, and `r2+` → 00.
-

`cmp <operand 1> <operand 2>`

Compares the values in the two specified registers and stores the result in the `less_than` flag if operand 1 is less than operand 2 using an unsigned comparison. Note that the comparison flag is persistent and only changes when the machine resets, or when the `cmp` instruction is executed.

```

#start
...
    //check for equality
    cmp r0 r1
    jumplt #not-equal

    cmp r1 r0
    jumplt #not-equal

    push
    call #run-if-equal
    pop

    #not-equal
...

#run-if-equal
...
    return

```

Format | Opcode 100 | Operand 1 (3 bits) | Operand 2 (3 bits) |

- Operand 1 specifies one of eight registers (r0 - r7)
- Operand 2 specifies one of eight registers (r0 - r7)

halt

Halts program execution and raises the done signal in the processor's output and enters a waiting state until the start signal is lowered.

```

#program-1
...
    halt

//wait for new operands to be loaded into memory
#program-2
...
#end

```

Format | Opcode 011 | 000111 |

iii. Internal operands

- a. 8 registers with an 8-bit width are supported
- b. Register 0 is the default output for most operations and serves as an accumulator. The other registers are all general-purpose. Registers 4-7 are always restored from the stack, but whether registers 1-3 are restored is configurable when pushing a new frame to the stack.

iv. Control flow

- a. Conditional branches using the less than comparison, and unconditional branches are both supported. Linked branches for function calls are also supported using the stack. A single program supports up to 63 branch locations (1 is reserved for function call returns).
- b. Target addresses are 10 bits each and stored as absolute positions as a separate section in instruction memory. The first 8 bits represent an instruction address, and then one bit determines whether the branch will be conditional, and another bit determines if the branch will be linked. The final bit is always zero, except for the **return** instruction and this particular address is always stored at the first label memory location.

The label memory section is stored in addresses 896-1023 in the instruction memory. Since instruction memory is 9 bits wide, each 18-bit label is stored as follows: Even addresses store the 9 most significant bits of the target address, and then the following odd address stores the least significant bit of the target address in the MSB, followed by the eight bits determining the type of branch, in the order (from MSB to LSB): {call, return, jumpz, jumpnz,

`jumppos, less_than, greater_than, signed`} where the `signed` bit indicates whether the comparison flags used should be signed or unsigned. If all eight bits determining the jump type are zero, the jump is unconditional.

For example, a conditional signed less than jump to target address `0b1010101010` with the machine code address 25 (`0b011001`) corresponds to the label memory addresses 946 ($896 + 25 \cdot 2$) and 947. Address 946 stores `0b101010101` and address 947 stores `0b00000101`.

There are two additional considerations made with this addressing scheme. The label memory addresses 896-897 are reserved for the `return` instruction and should be the only entry with the `return` bit set to 1. This scheme also requires that for a given label, only one type of condition can be used to jump to it. To jump to a location using two different jump instructions, create another label on an adjacent line and both will be assembled into two jumps pointing to the same instruction.

- c. The maximum branch distance supported is only limited by the size of the instruction memory, which has 896 entries, so the maximum distance is 895 instructions.
- v.
 - a. Only indirect addressing is supported.
 - b. Addresses are calculated using the value in a given register and an immediate offset in the range of 0-3 bytes, allowing for a 4-byte word to be read without needing to calculate a new address.
 - c. Example:

```
#start
... //assume r3 holds 132 and 132-135 hold 0xABCDEF55
    load r3
    copy r4    //0xAB
    load r3 1
    copy r5    //0xCD
    load r3 2
    copy r6    //0xEF
    load r3 3
    copy r7    //0x55
```

4 Programmer's Model

- i. A programmer should think about operating the machine as a back-and-forth movement of data from the accumulator in register 0 to the other registers. Complete all operations using that result in register 0 before copying back to the destination to avoid copying more than needed.

Since register count is limited, the stack should be used to access a few additional temporary registers. Breaking apart code into smaller functions will facilitate this since saving to the stack is faster than calculating memory addresses to store and read from. When saving registers to the stack, limit the number of saved registers to only save the minimum number of registers needed. Since the register file is restricted to reading two registers and writing one register per cycle, saving n registers requires $\lceil n/2 \rceil$ cycles to push and n cycles to pop.

When making function calls, note that functions can return a maximum of 6 values by pushing `push r6+` and saving the return values in registers `r0` through `r5`. Also note that even if no registers need to be preserved after the function call is returned, the `push` instruction is still required beforehand to create a new stack frame since this instruction is required to save the return address since the `call` instruction only handles the jump without performing any stack operations.

- ii. Instructions cannot be directly copied from MIPS or ARM due to the instruction width differences resulting in missing fields for source and destination registers.
- iii. The ALU is used for some non-arithmetic instructions. Data that is passed in for `copy` and `imm` is output by the ALU unchanged. It is also used to calculate addresses given a base address and offset for memory accesses. However, it is not used to calculate the program counter. The next program counter is determined only by the program counter module or the stack controller. Separating the ALU use this way simplifies design so that bypasses for loading immediates or copying are not needed for those instructions. It also avoids extra complexity in determining the next program counter since there are already multiple modules providing the next program counter to be used.

iv. Machine code example of the earlier equality checking example:

```
...                //(start)
[5] 001 000 001 //cmp r0 r1
[6] 000 000001 //jumplt #not-equal-lt
[7] 001 000 001 //cmp r0 r1
[8] 000 000001 //jumpgt #not-equal-gt (different label needed for different jump type)
[9] 011 0 11 000 //push r4+
[10] 000 000010 //call #run-if-equal
[11] 011 100000 //pop
[12] ...        //(not-equal-lt)
                //(not-equal-gt)
[30] 001 000000 //return

//Address memory begins
[896] 000000000 001000000 //(return)
[898] 000000000 000000000 //(start)
[900] 000001100 000000100 //(not-equal-lt) conditional branch
[902] 000001100 000000010 //(not-equal-gt) conditional branch
```

5 Individual Component Specification

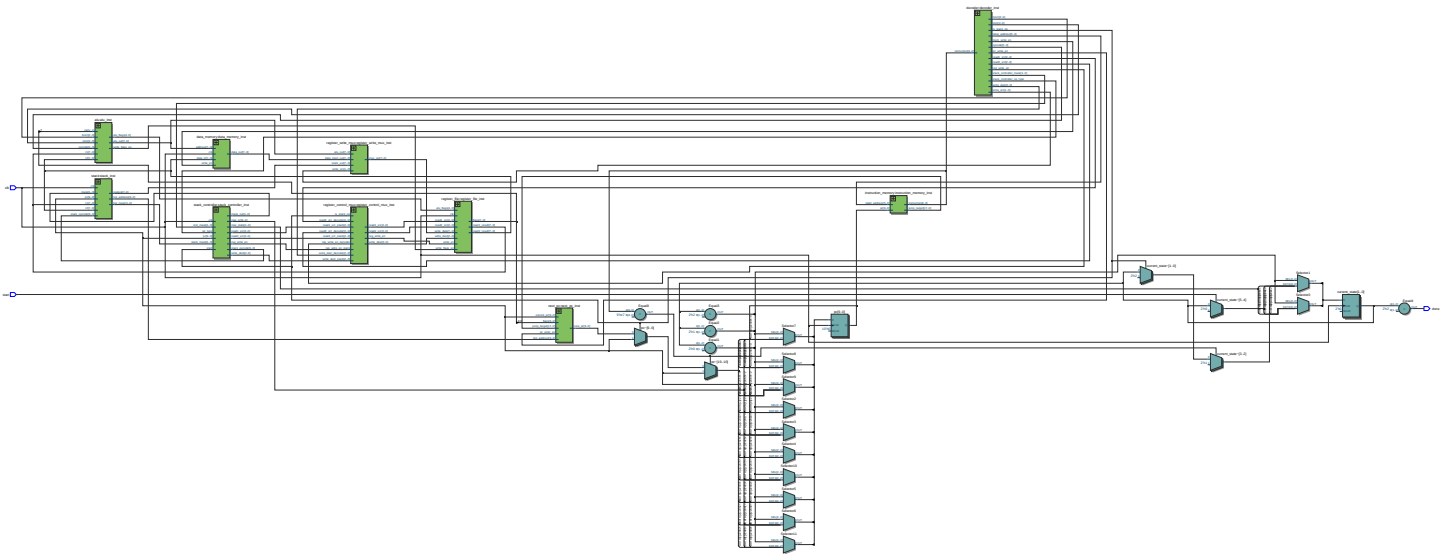
a. Top level

Name: `top.sv`

Functionality: This is the top-level module for a single-cycle processor. The top-level module has a state machine with three states: exec, halted, and stack, and contains the 10-bit register storing the current program counter.

The machine starts in the halted state and remains there until the start input is raised, at which point it enters the exec state. In the exec state, one instruction is executed every cycle. If a stack operation (**push/pop**) is reached, the machine enters the stack state. This stack state hands over control of the program counter, register file, and top-level state to the stack controller. When stack operations are finished, the machine returns to the exec state. When a **halt** instruction is executed, the machine returns to the halted state and raises the done signal.

Schematic:



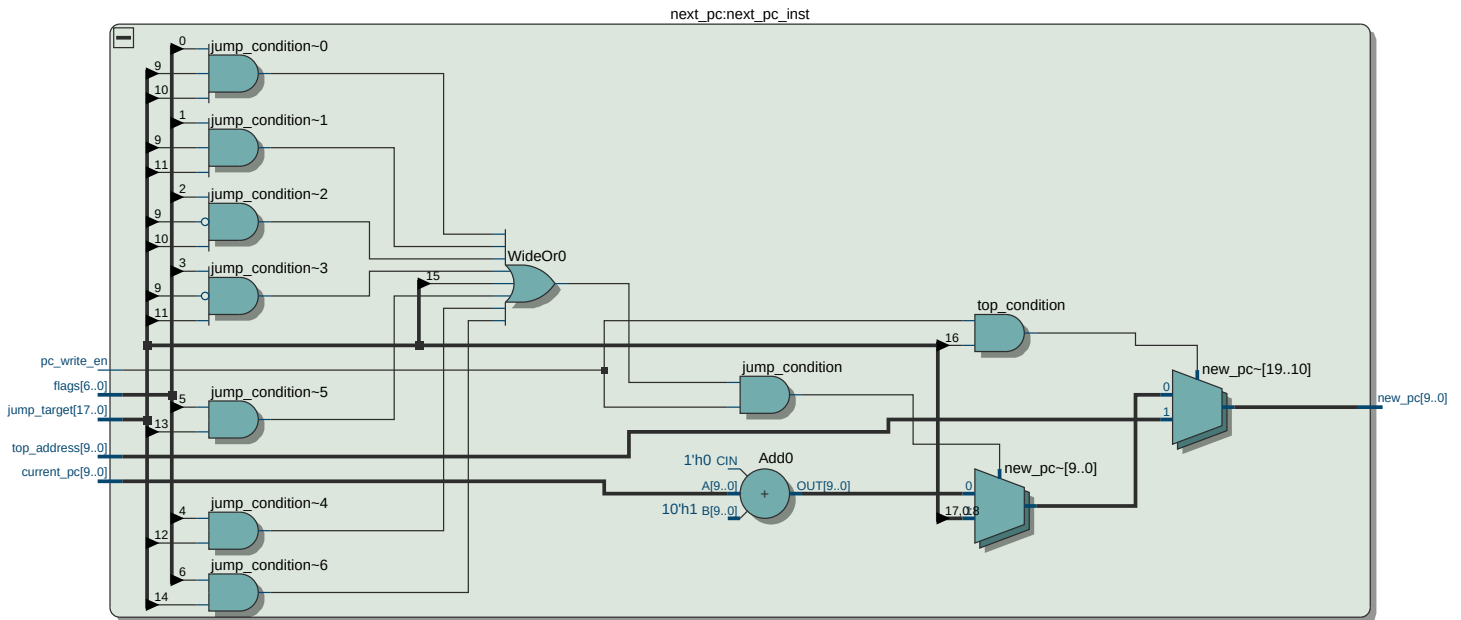
b. Program counter

Name: `next_pc.sv`

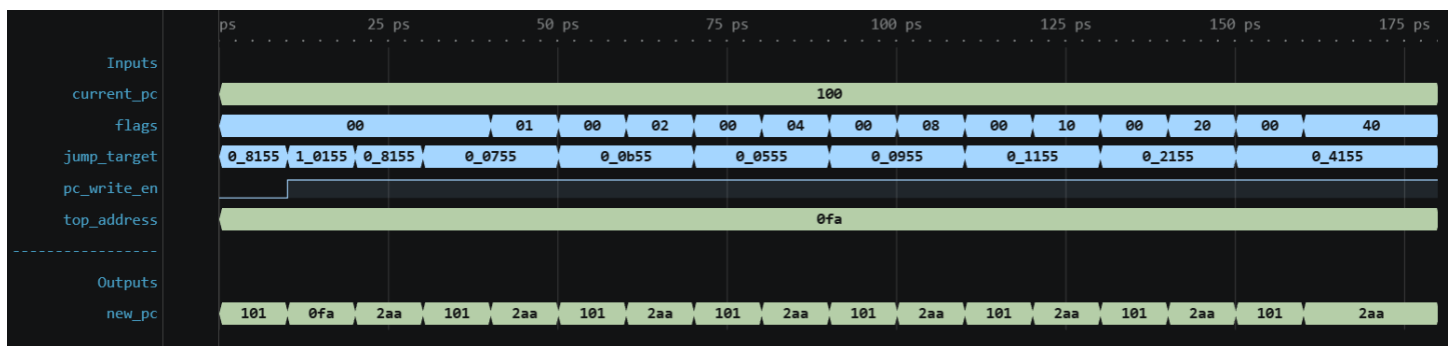
Testbench name: `next_pc_tb.sv`

Functionality: This module determines the next program counter value. In most cases, the program counter is assigned to the current program counter plus one. If any jump instruction is being executed, the flags stored in the register file are compared with the corresponding flags from label memory to determine if the next program counter should be changed.

Schematic:



Timing diagram:



Tested with current program counter at 100, jump target at 0x2AA, and top program counter in stack at 0x0FA. Branch/jump operations tested:

- No change (program counter + 1)
- Pop from stack
- Unconditional
- Greater than (signed) not taken
- Greater than (signed) taken
- Less than (signed) not taken
- Less than (signed) taken
- Greater than (unsigned) not taken
- Greater than (unsigned) taken
- Less than (unsigned) not taken
- Less than (unsigned) taken
- Positive not taken
- Positive taken
- Nonzero not taken
- Nonzero taken

p. Zero not taken

q. Zero taken

c. Instruction memory

Name: `instruction_memory.sv`

Functionality: This module stores the instruction memory and label memory. The memory has 1024 entries, each 9 bits wide. At the bottom of the memory where labels are stored, an additional port outputs the current label's flags and target address.

Schematic:

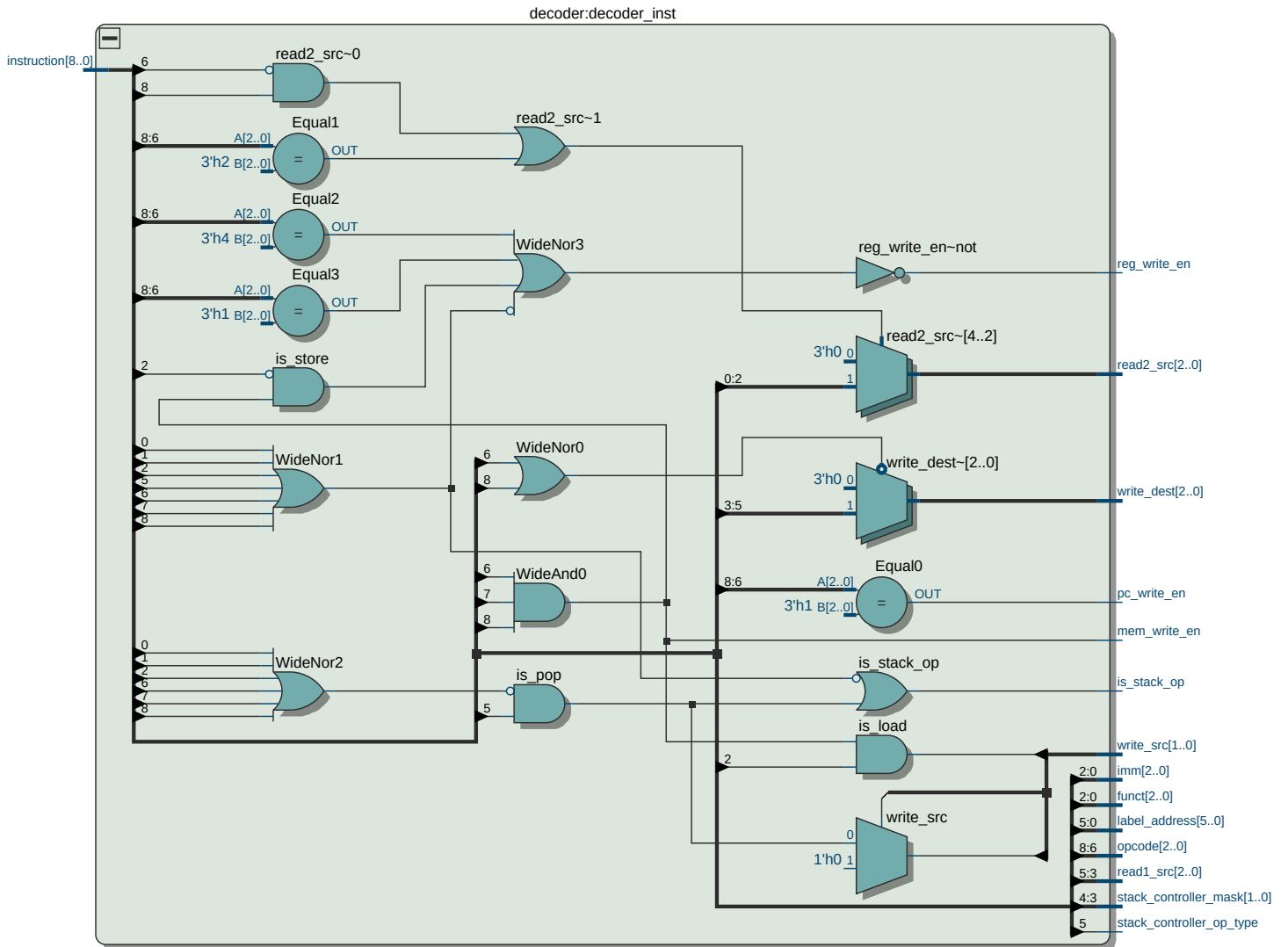


d. Control decoder

Name: `decoder.sv`

Functionality: This module determines the control signals for the ALU, label memory, data memory, register file, next program counter, stack controller, and multiplexers.

Schematic:

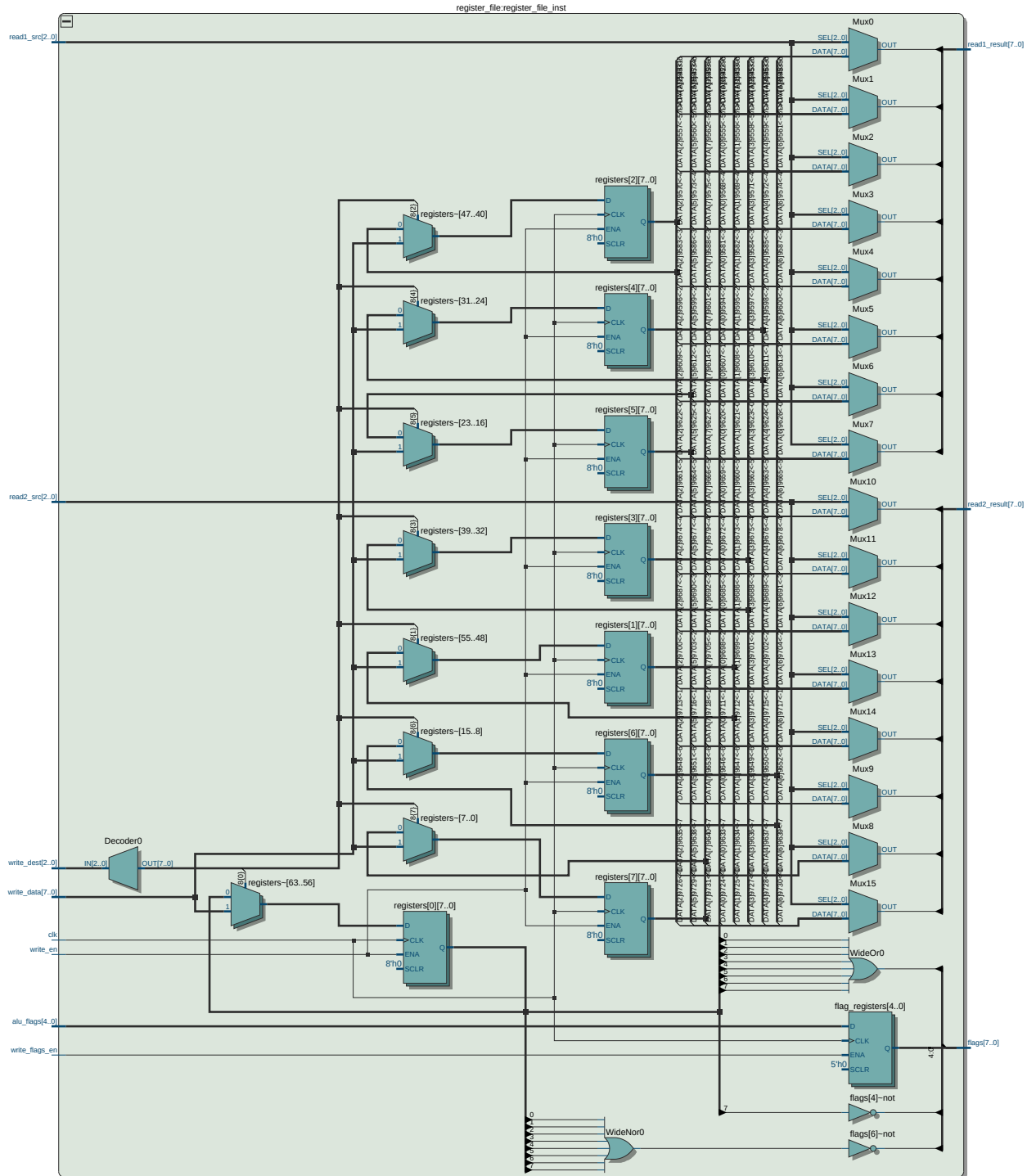


e. Register file

Name: register_file.sv

Functionality: This module holds eight 8-bit registers and the current flags. The four comparison flags and carry flag are set by the ALU, while the other three are set depending on the state of the zero register. The five flags set by the ALU are modified only by instructions that produce them. Values are read combinatorially, while writes are completed at the rising edge of the clock.

Schematic:



f. ALU

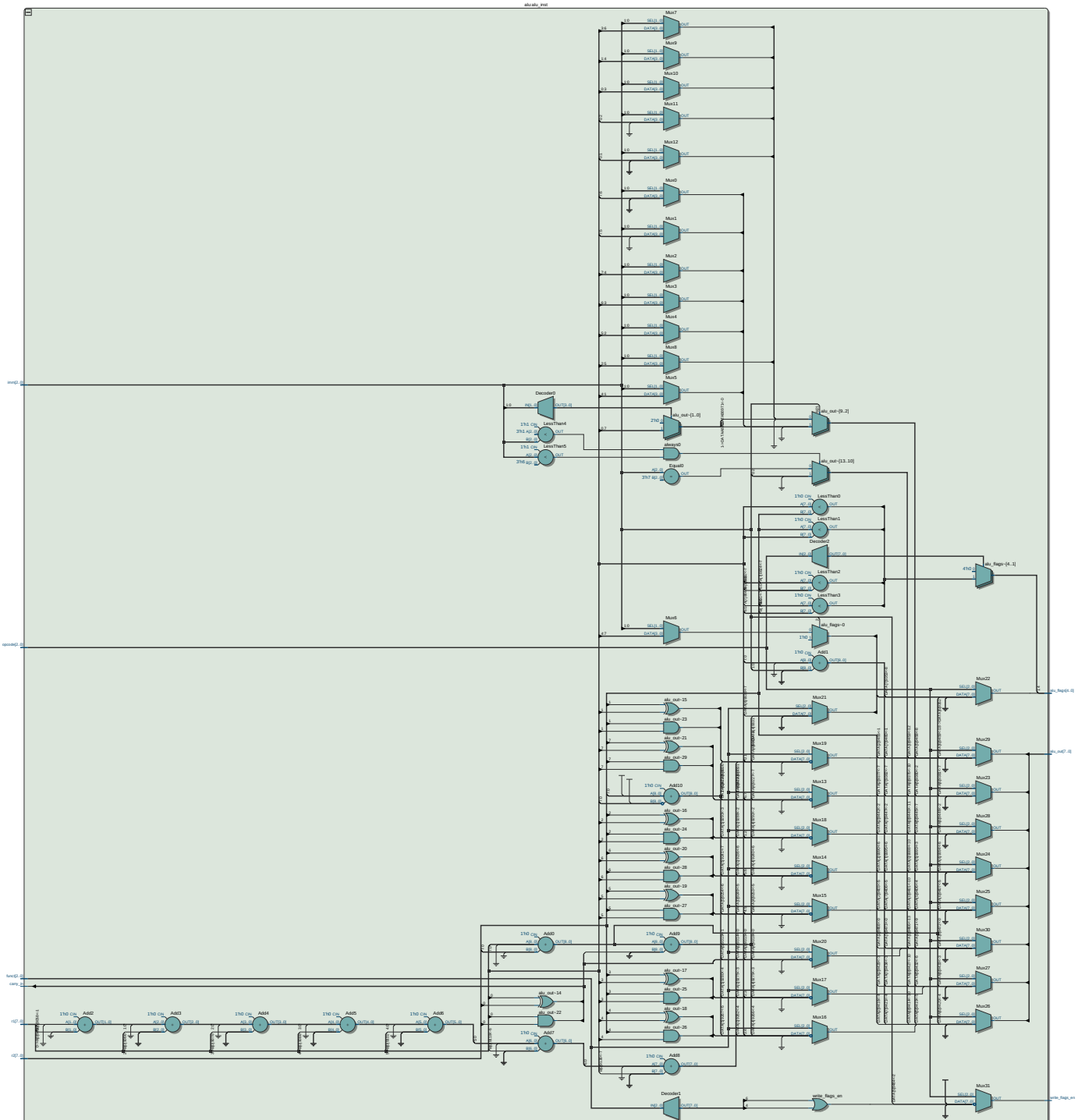
Name: `alu.sv`

Name: `alu_tb.sv`

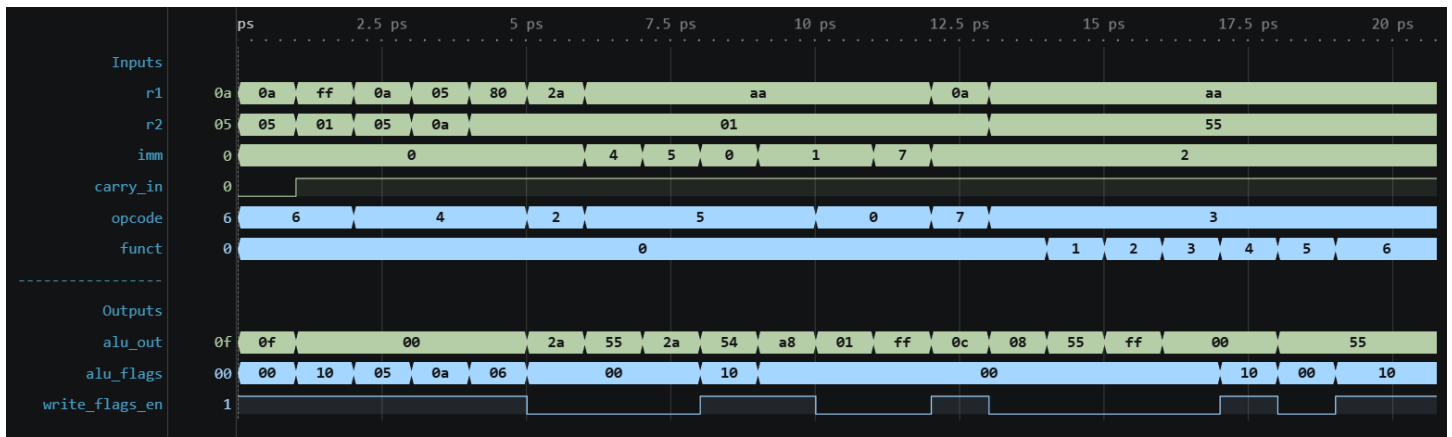
Functionality: This module performs arithmetic for instructions and calculates memory addresses. Two register values and an immediate are passed in from the register file and decoder, and the ALU chooses from them as needed. Supported operations include addition, subtraction, AND, XOR, NOT, population count, and shifts. All operations pass through the ALU, including the `copy` instruction.

The carry flag is set by additions and left shifts, while the comparison flags are set by the `cmp` instruction. All flags are set at the same time as each other (if the carry is set, then the comparison flags are reset, and vice versa).

Schematic:



Timing diagram:



Operations being tested in the timing diagram:

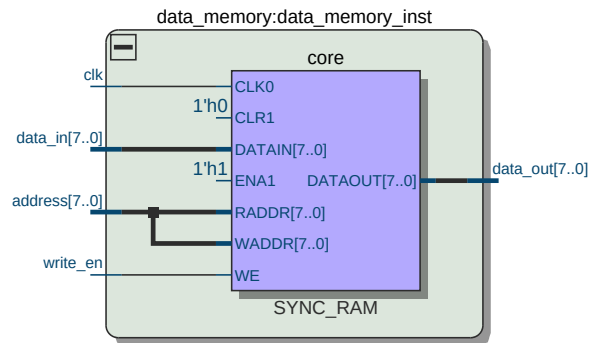
- 110: 0x0A + 0x05
- 001: 0xFF + 0x01
- 001: cmp 0x0A, 0x05 using different signedness and orderings
- 010: 0x2A => 0x2A copy
- 101: 0xAA >> 1, 0xAA >> 2, 0xAA << 1, 0xAA << 2
- 000: 0x01, 0x07 immediates
- 111: 0x0A + 0x02 address + offset
- 011: Single-operand instructions
 - 000: popcnt(0xAA)
 - 001: ~0xAA
 - 010: 0xAA ^ 0x55
 - 011: 0xAA & 0x55
 - 100: 0xAA + 0x55 + carry
 - 101: 0xAA - 0x55
 - 110: (0xAA << 1) | carry

g. Data memory

Name: `data_memory.sv`

Functionality: This module holds a data memory with 256 entries, each with 8 bits. Values are read combinationally, while writes are completed at the rising edge of the clock.

Schematic:



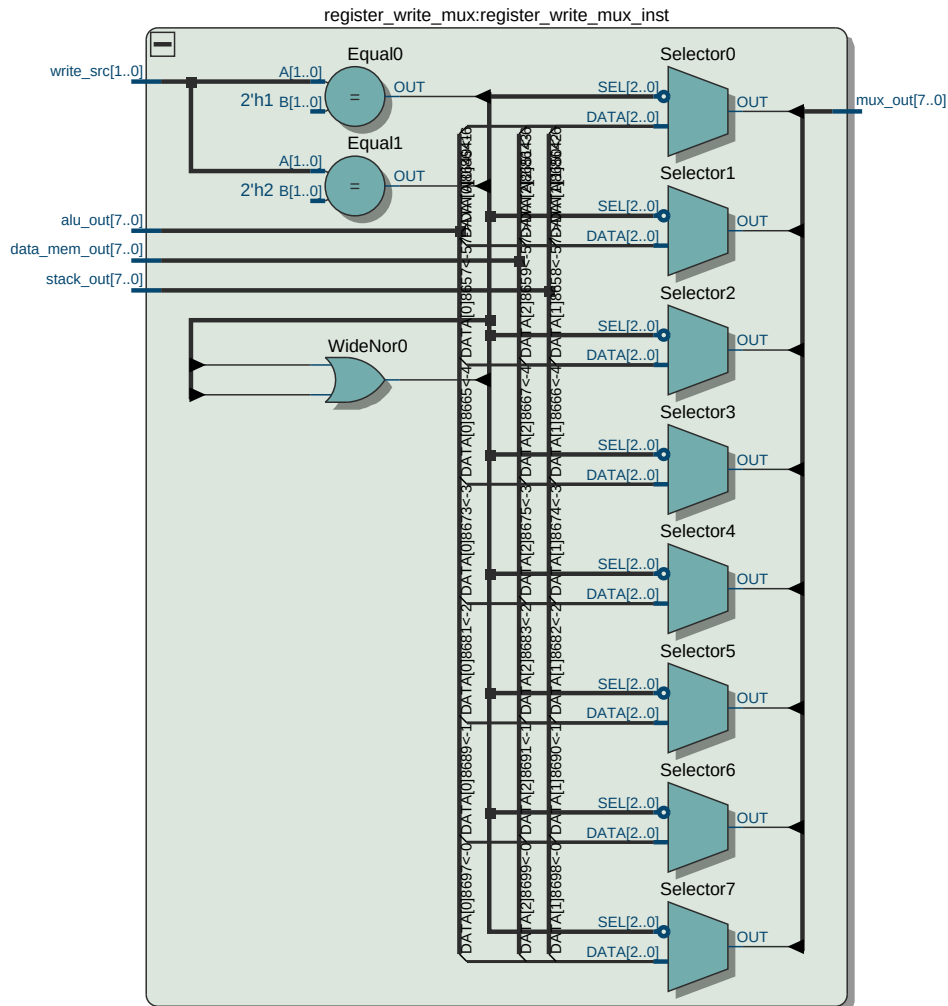
h. Lookup tables: none

i. Register write MUX

Name: `register_write_mux.sv`

Functionality: This module contains a three-input MUX for switching between the ALU result, data memory, and stack to determine which data is written to the register file.

Schematic:

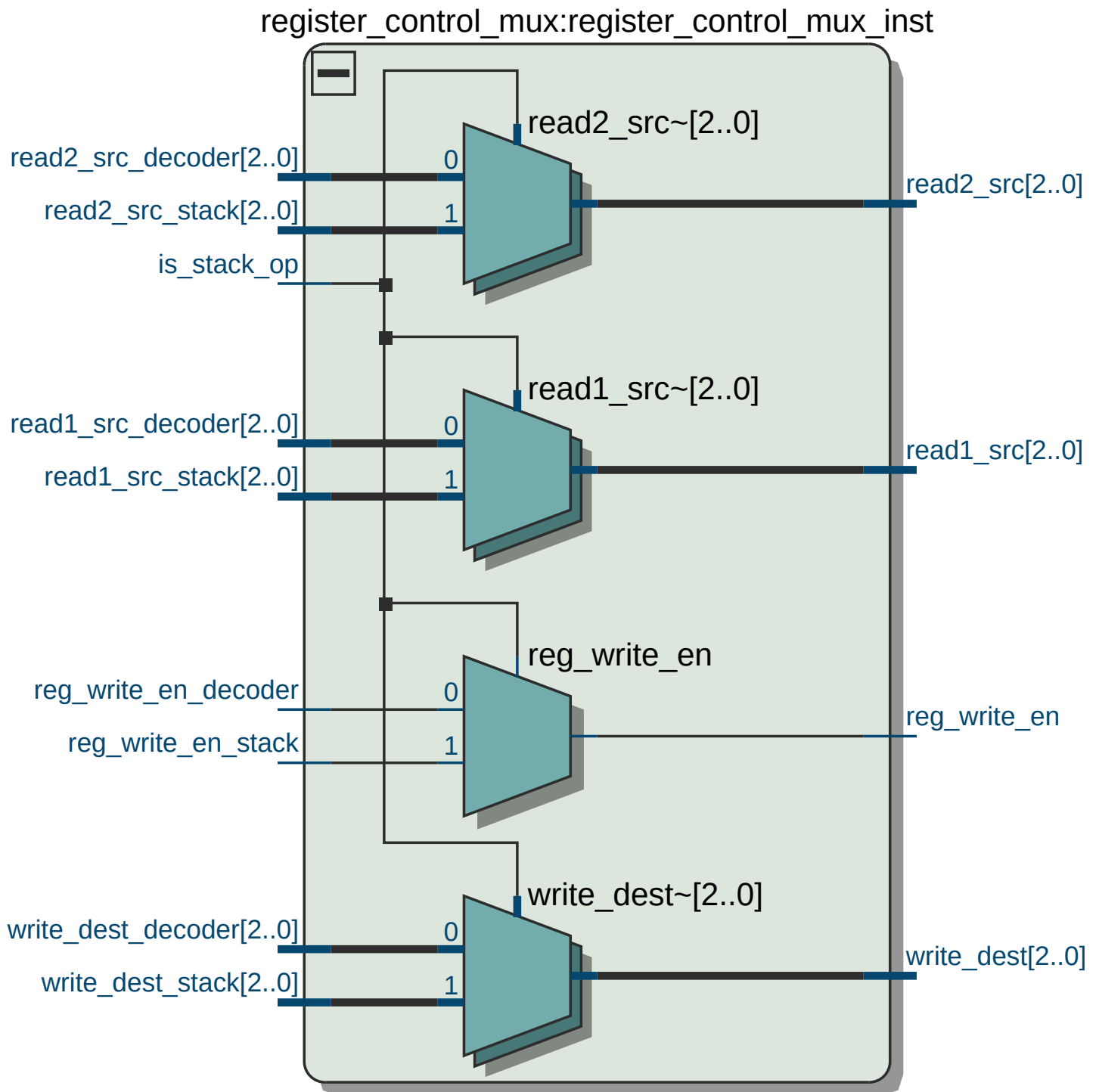


i. Register control MUX

Name: `register_control_mux.sv`

Functionality: This module contains a two-input MUX for switching between the decoder and stack controller to determine which control signals are sent to the register file.

Schematic:



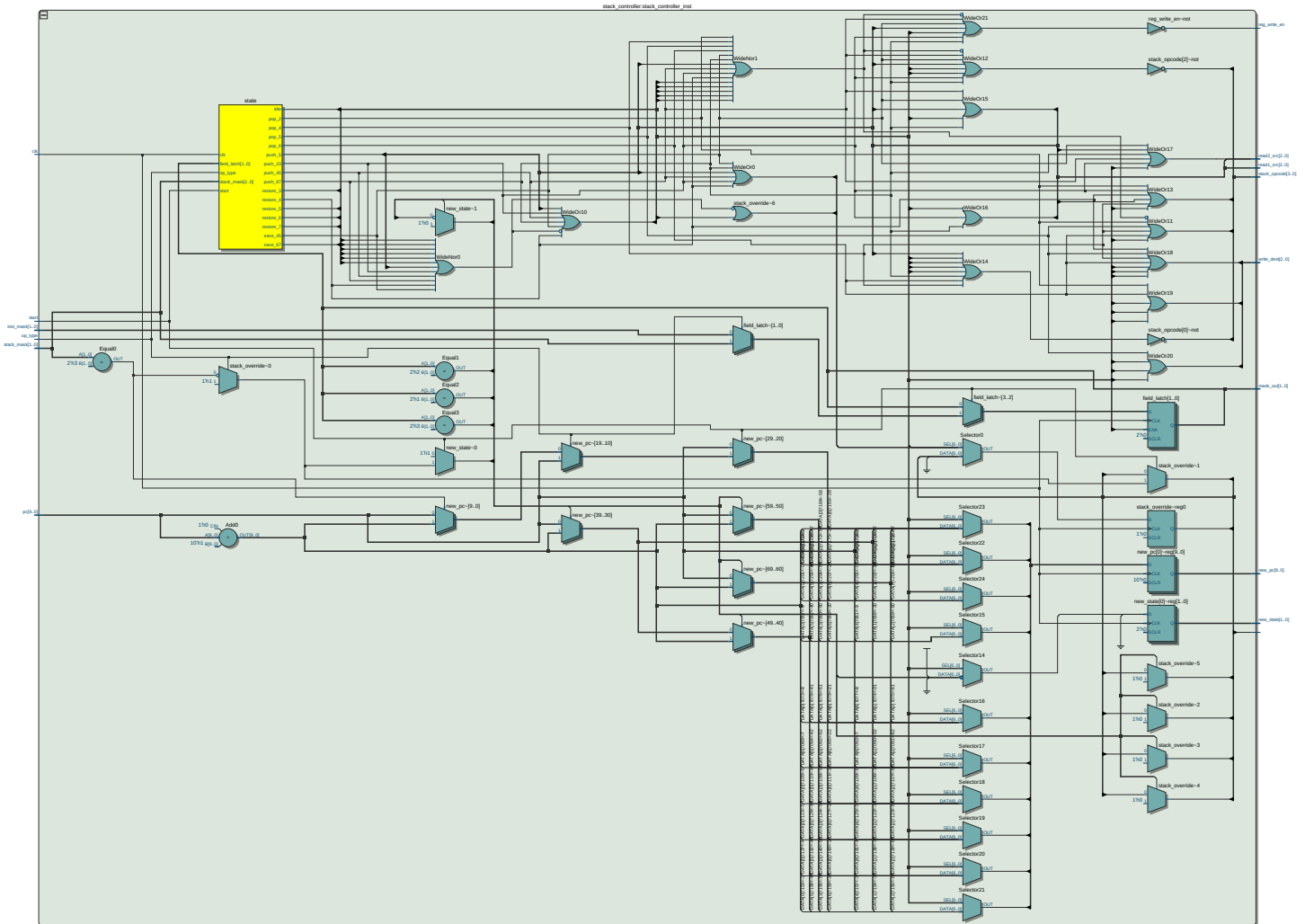
j. Stack controller

Name: `stack_controller.sv`

Functionality: This module is a Moore state machine that produces a sequence of registers to read or write to when receiving a stack operation. It also generates a stack opcode to determine what operations should be performed by the stack module. While the stack controller is not idle, it generates a next program counter and next top-level module state which overrides the typical behavior.

Pushing n registers to the stack takes $\lceil n/2 \rceil + 1$ cycles while popping n registers from the stack takes $n + 1$ cycles.

Schematic:

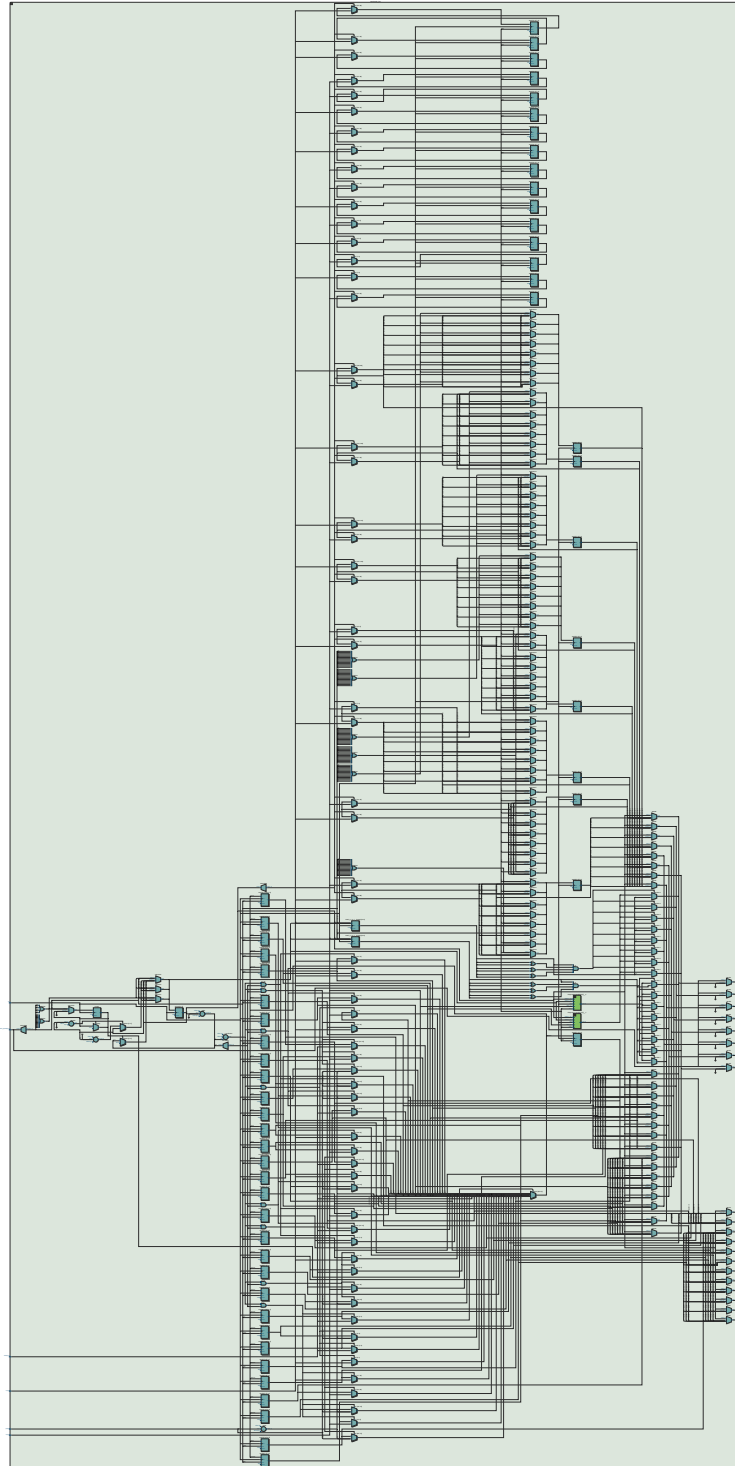


k. Stack

Name: `stack.sv`

Functionality: This module stores up to eight stack frames, with a pointer keeping track of the current entry at the top. Each stack frame is made up of six 8-bit registers, a 2-bit mask code, and a 10-bit program counter. When a stack is popped, the mask code is used to determine which registers should be restored. Values from the stack are read combinationally (when popping) and written at the positive edge of the clock (when pushing).

Schematic:



j. Types

Name: `types.sv`

Functionality: Not a module, but contains a package with definitions for state enums and stack frames.

6 Program Implementations

Each program is included separately as `hamming.asm`, `pairs.asm`, and `multiply.asm`. Their respective `.py` and `.mif` files contain pseudocode demonstrating their algorithms and the commented machine code.

The assembler is included as `assembler.py` and takes three command line arguments. The first is the name of the file containing the assembly code, the second is the `-o` option to specify the name of the output machine code file, and the third is the `-b` flag to output a binary text file or a memory initialization file with comments.

1. The first program uses a nested loop to iterate through the bits in memory. The Hamming distance of each unique pair is calculated using the XOR and the population count instructions. This distance is then compared with the current largest and smallest distances seen so far. Once the loop is finished, the largest and smallest distances are written to memory.
2. The second program first sorts the memory using a recursive merge sort algorithm. At each recursive step, the two merged subarrays are copied into a region in memory offset by 64 entries to perform the merge, before being copied back into the original memory addresses to create a result that sorts the array in place.

Once the sorting is finished, the smallest and largest numbers at the ends of the array are subtracted to get the largest difference. The smallest difference is then calculated by iterating through adjacent pairs in memory and keeping the smallest one. These results are then written to memory.

3. The third program uses an 16-bit unsigned multiplication function. At each step of multiplication, the partial product is shifted left and incremented as needed. These operations store partial products in memory due to limited register space. Since the memory read/write offsets are available up to 3, fewer pointers are needed to keep access to all the current data.

After the unsigned multiplication is completed, the product is negated based on the sign calculated at the start of each loop iteration before being written to memory.

7 Changelog

Milestone 1 → 2

0. Added SystemVerilog modules and descriptions.
1. Fixed typos in the example given for the `call` instruction: wrong syntax for push/pop instructions
3. Replaced the `call` bit with the `unconditional` bit in odd-indexed label memory entries and switched the position of this bit with the `return` bit to reduce program counter branching logic. Machine code and the assembler are changed accordingly.
4. Included what flag bits are stored in the register file.
5. Replaced the `dist` instruction with the RISC-style `popcnt` instruction and updated the description for `imm` to use the `xor` to load zero immediates.
6. Fixed typos in the field descriptions for `load` and `store` describing addresses as operands.
7. Introduced the stack controller module to interface between the decoder and the stack.

Milestone 2 → 3

0. Updated program implementations and machine code
1. Added assembler

Milestone 3 → 4

0. Updated testbenches and associated shell scripts.
1. Made logic fixes in the stack's register read combinational block.
2. Added assembler safeguards against loading immediate zeroes.

3. Changed the `save_45` state in the stack controller's state machine to correctly exit 2 cycles later.
4. Fixed incomplete logic for the `mem_write_en` control signal in the decoder.
5. The top-level state machine no longer resets the program counter after being halted so that a combined program can be run.
6. Renamed top module to `DUT` and data memory instantiation to `dm`.
7. Split apart two states in the stack's register restoration state machine to correctly restore registers 2 and 3.
8. Corrected address calculation for storing results in `pairs.asm`.
9. Increased stack depth to 16 to avoid stack overflows during sorting.
10. Reversed the logic tracking the validity of the stack to avoid initialization issues.
11. The top-level state machine now correctly responds to the negative edge of the start signal instead of the positive edge.
12. Disabled writing ALU flags to the register file for memory instructions and removed inferred latch on `alu_flags` and `write_flags_en`.
13. Fixed assembler output for immediate values in shift instructions.
14. Added a cycle counter to the emulator and top-level module for comparison and debugging.